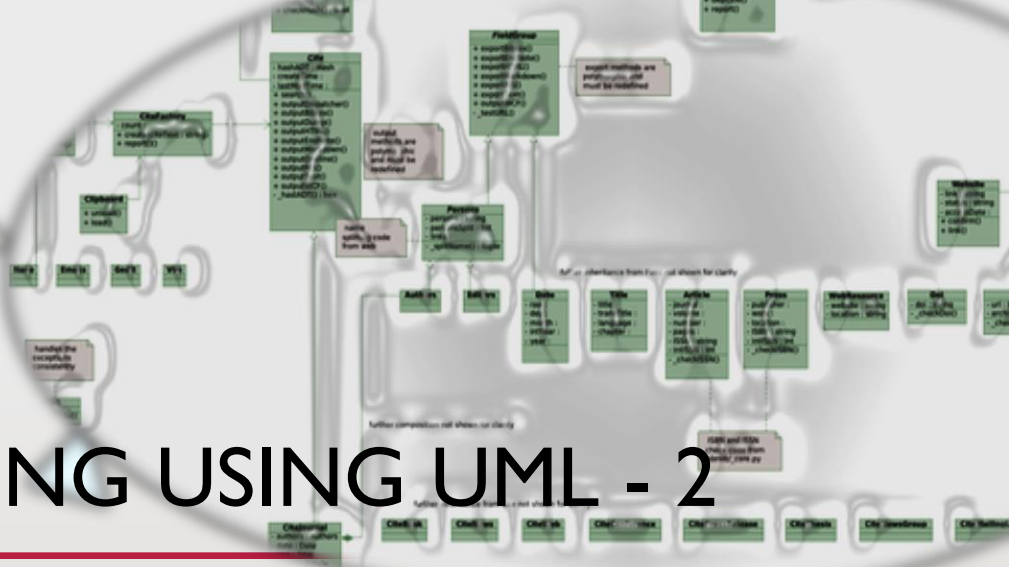




COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



PREVIOUS EXERCISE

```
public abstract class Shape {
    protected int _x;
    protected int _y;
    protected int _deltaX;
    protected int _deltaY;
    protected int _width;
    protected int _height;
    ...

    public void move( int width, int height ) {
        ...
    }

    public abstract void paint( Painter p );
}
```

```
public class RectangleShape extends Shape {
    ...
    public void paint( Painter p ) {
        ...
    }
}
```

```
public class OvalShape extends Shape {
    ...
    public void paint( Painter p ) {
        ...
    }
}
```

```
public interface Painter {
    ...
    void drawRect( ... );
    void drawLine( ... );
    void drawOval( ... );
}
```

```
public class GraphicsPainter implements Painter {
    private Graphics _g;
    ...
}
```

```
public class MockPainter implements Painter {
    ...
}
```

*DRAW A CLASS DIAGRAM
FOR SHAPES APP CODE
(SOLUTION ON CANVAS)*

```
public class AnimationViewer extends JPanel
    implements ActionListener {
    private List<Shape> _shapes;
    private Timer _timer = ...
    ...

    public void paintComponent( Graphics g ) {
        ...
    }

    public void actionPerformed((ActionEvent e) ) {
        ...
    }

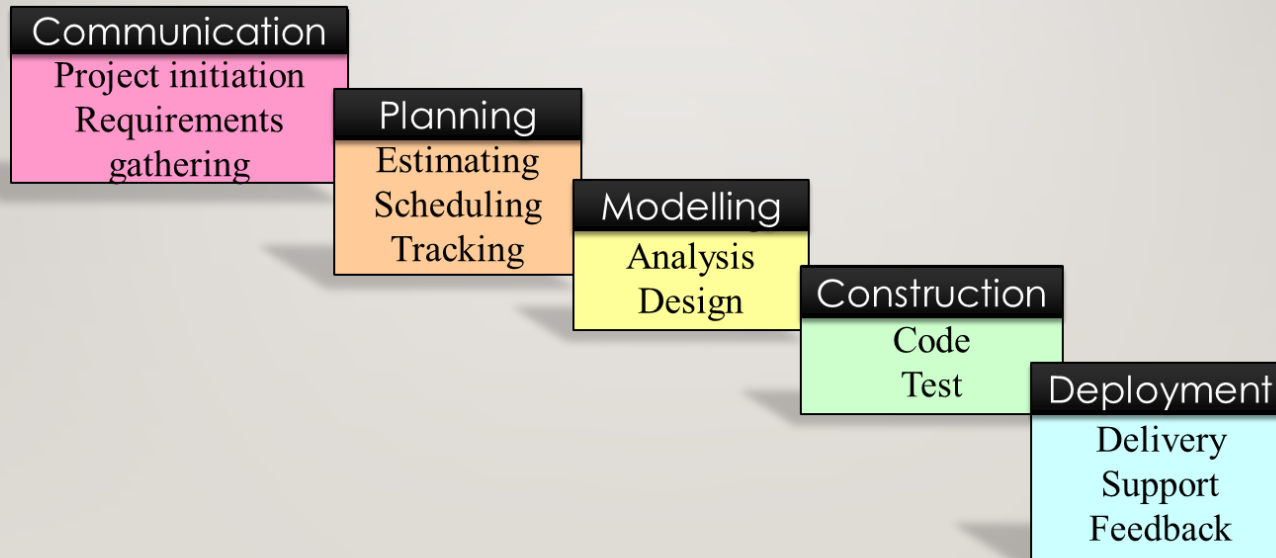
    public static void main( String[ ] args ) {
        JFrame frame = ...
        ...
    }
}
```

AGENDA

- *Admin*
- More on class diagrams
 - Multiplicity constraints
 - Aggregation
 - Composition
- Object diagrams
- Sequence diagrams
- Well-formed diagrams

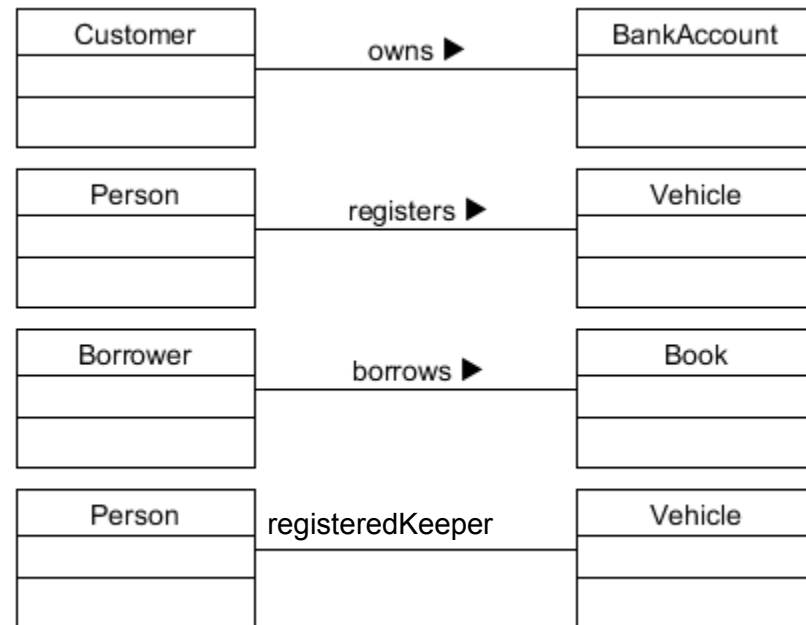
THE LARGER PICTURE

SOFTWARE ENGINEERING PROCESS



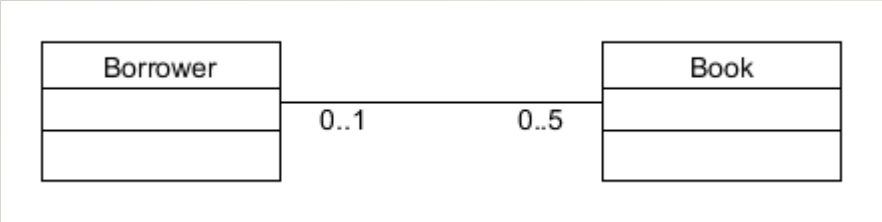
MORE ON CLASS DIAGRAMS

- Class diagrams can be made more precise through:
 - Decorating associations with **names** and **roles**
 - Specifying association **multiplicities**
 - Using **aggregation** and **composition** relationships, where appropriate, instead of (simple) association



MORE ON CLASS DIAGRAMS

- **Association multiplicities**
 - Multiplicity is a constraint on the number of instances of one class that can be related to one instance of the other class



- A Book object can be associated with at most one Borrower instance.
- A Borrower object can be associated with up to 5 Book instances.

Many	*
Exactly one	1
Zero or more	0..*
One or more	1..*
Zero or one	0..1
Specified range	2..4


```
public class Borrower {  
    private static final int BORROWING_CAPACITY = 5;  
    private List<Book> _booksOnLoan;  
    ...  
    public boolean canBorrow() {  
        return _booksOnLoan.size() < BORROWING_CAPACITY;  
    }  
  
    public boolean borrow( Book b ) {  
        if ( ! canBorrow() ) || ( b.isOnLoan() ) {  
            return false;  
        }  
        _booksOnLoan.add( b );  
        b.setBorrowedBy( this );  
        return true;  
    }  
    ...  
}
```

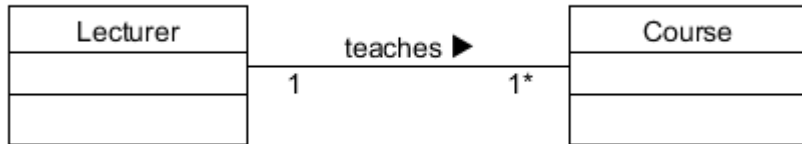
```
public class Book {  
    private Borrower borrowedBy;  
  
    public Book( ) {  
        borrowedBy = null;  
    }  
  
    void setBorrowedBy( Borrower b ) {  
        borrowedBy = b;  
    }  
  
    public boolean isOnLoan( ) {  
        return borrowedBy != null;  
    }  
    ...  
}
```

Why isn't Book's setBorrowedBy() method public?

MULTIPLICITY CONSTRAINTS



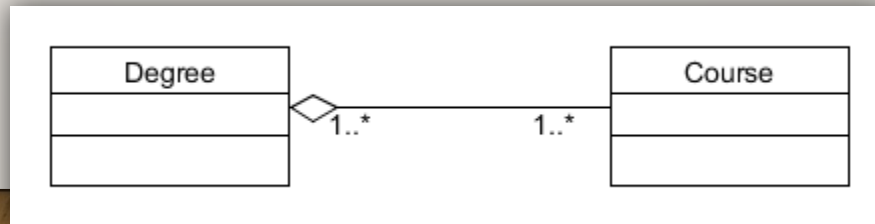
What multiplicity indicators are required to express that a Vehicle must have one registered keeper and that a Person may register any number of Vehicles?



Can a Lecturer be on sabbatical?
Can teaching of a Course be shared by Lecturers?
Would a Lecturer be happy with this model?

AGGREGATION RELATIONSHIPS

- Aggregation is a special form of association that **represents containment**
 - Aggregation is also known as the **has-a** relationship
 - Like an association:
 - An aggregation can be bi-directional or unidirectional
 - An aggregation can be decorated with multiplicity constraints
- The distinction between association and aggregation is conceptual
- Aggregations are implemented in code just like associations (i.e. as instance variables)

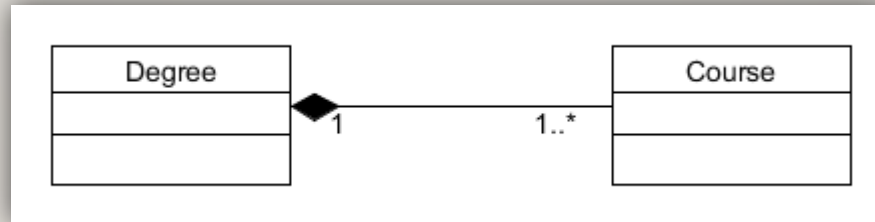


Class playing the **whole** role

Class playing the **part** role

COMPOSITION RELATIONSHIPS

- Composition is a **more restrictive form of aggregation**
- With a composition:
 - A **part object cannot be shared** by multiple whole objects
 - The part object is often **fully encapsulated** by its whole
 - In this case, the only object that refers to the part object is its whole object
 - If the **whole instance is copied/deleted**, so are its parts



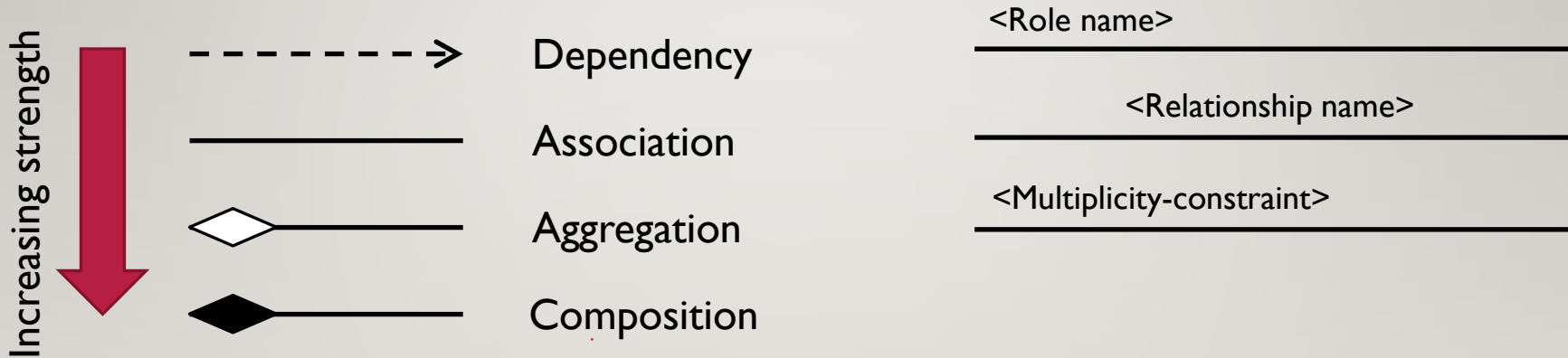
Class playing the **whole** role

Class playing the **part** role

DISTINGUISHING FACTORS AMONG ASSOCIATION TYPES

- **Scope** (of referenced object)
 - defines the part of a class body where an instance of an outside class may be accessed.
- **Ownership** (of referenced object)
 - an object *objA* of class *A* owns an object *objB* of class *B*, if *objB* is "created" inside *objA*
- **Sharing** (of owned object)
 - an object *objB* of class *B* can be accessed/shared by two or more different objects
- **Lifetime** (of associated objects)
 - the lifetime of an object *objA* of class *A* is defined by its existence in memory, i.e. the moment it is created in computer memory until it is destroyed.

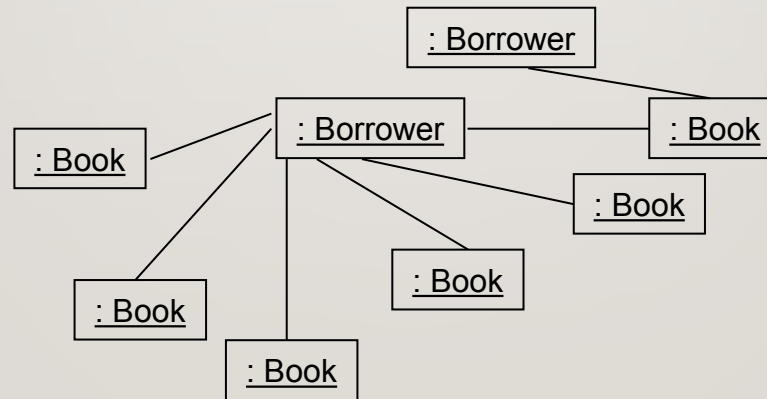
RELATIONSHIPS AND DECORATIONS



A QUICK EXERCISE?

OBJECT DIAGRAMS

- Another of the UML's diagrams is the **object** diagram
- An object diagram captures a snapshot of a system's objects and their connections at a particular point in time

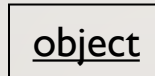


OBJECT INTERACTION DIAGRAMS

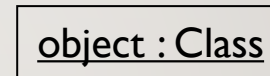
- **Object interaction diagrams** show how class instances behave in terms of sending messages to one another

- **Notation:**

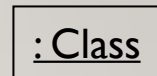
- **Object**



An instance, named “object”, of an unspecified class

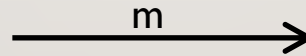


An instance, named “object”, of a class named “Class”

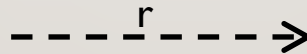


An unnamed instance of class “Class”

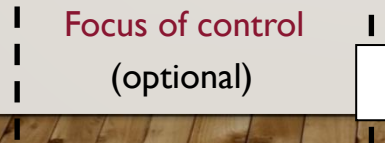
- **Message m**



- **Message reply r (optional)**

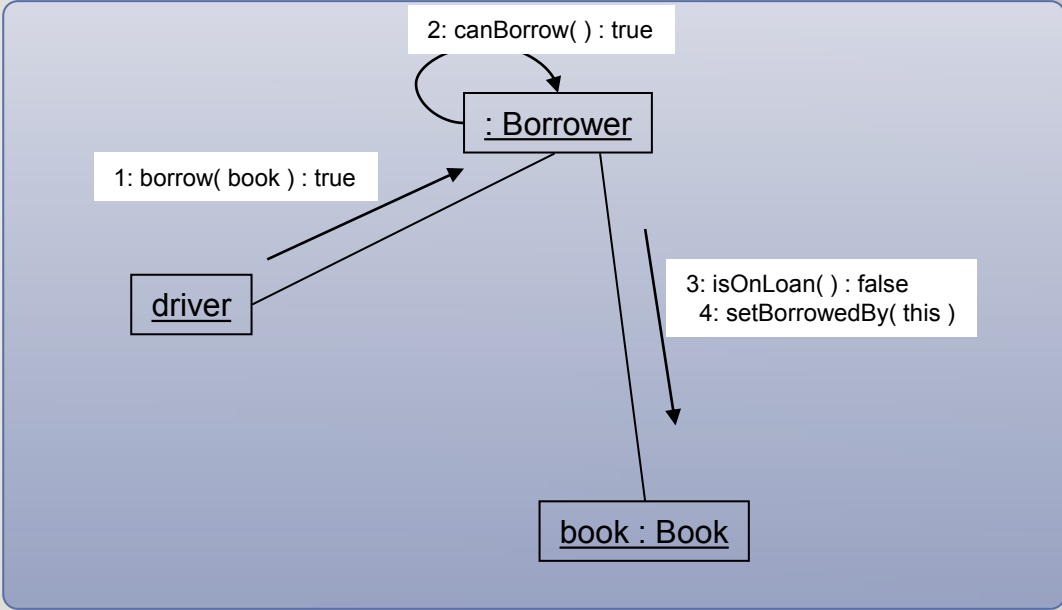
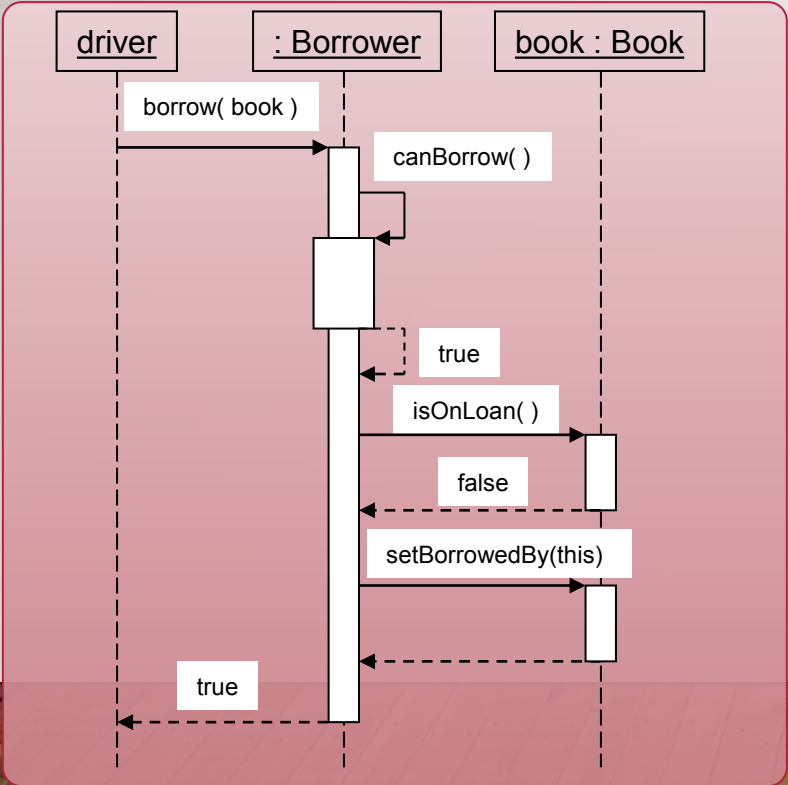


- **Object lifetime**



Focus of control
(optional)

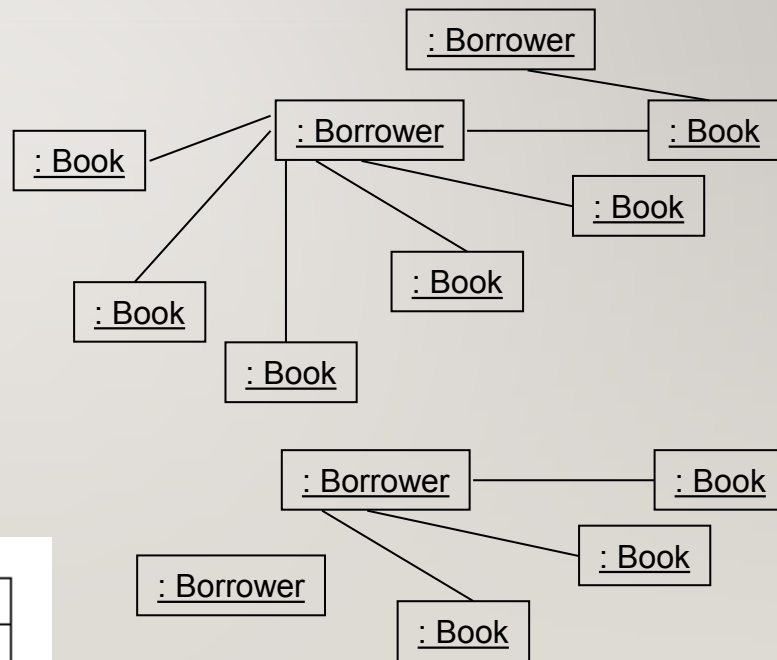
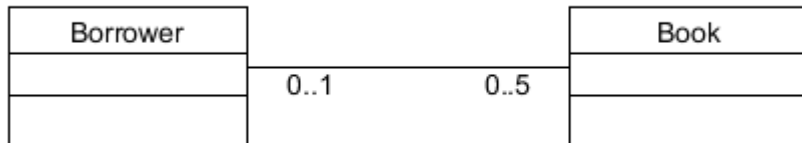
A **sequence** diagram is a popular form of interaction diagram where messages are time-ordered



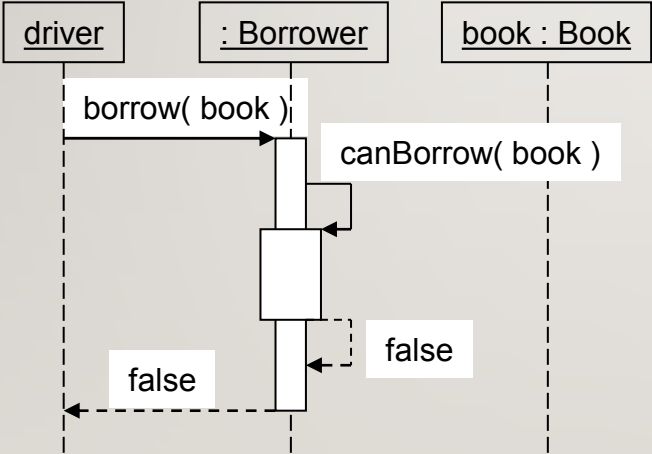
Collaboration diagrams are semantically equivalent to sequence diagrams but emphasise connectivity between objects

WELL-FORMED MODELS

- A model comprises a collection of views (different types of diagrams)
- A **well-formed** model is one whose views are semantically self-consistent with one another
- In practice, less than well-formed models are often used
 - Certain elements may be hidden for simplicity
 - Views may be inconsistent as a design develops over time

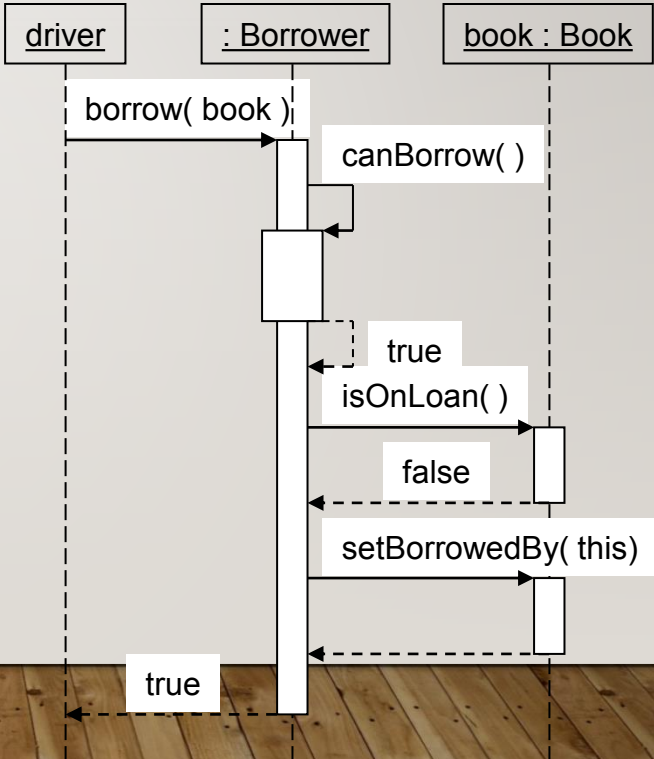


WELL FORMED MODELS



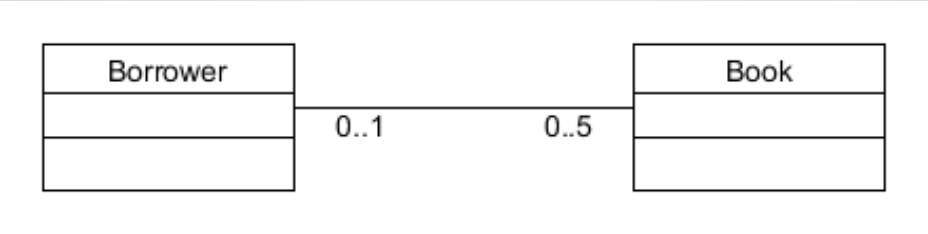
A scenario whereby a Borrower is not eligible to borrow.

A scenario whereby a Borrower is eligible to borrow a book that is not on loan. The borrower successfully borrows the book.



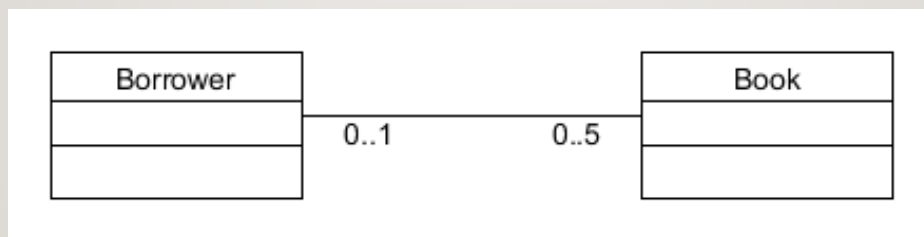
EXERCISE 1A

Write at least two interpretations of the following class diagrams in terms of the relationship between objects of the classes shown.



EXERCISE 1A (SOLUTION)

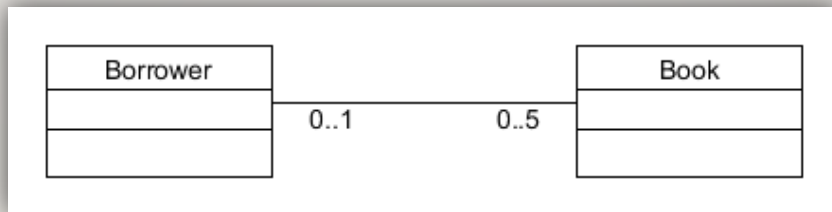
Write at least two interpretations of the following class diagrams in terms of the relationship between objects of the classes shown.



- A borrower can borrow at the most five books
- A book can not be borrowed by more than one borrower

EXERCISE 1B

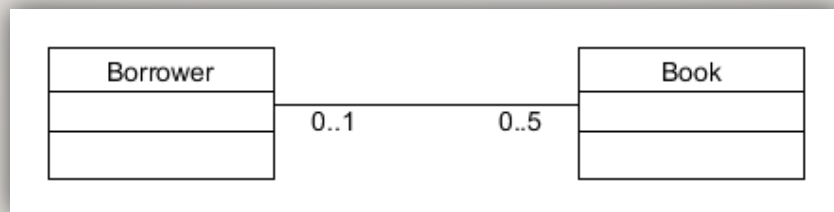
Which of the following interpretations about the class diagrams shown below is TRUE?



- a. Each borrower must have at least one book issued
- b. Each book may have less than one borrower
- c. Each borrower may have 6 books issued to their name
- d. Each book must have at least one borrower

EXERCISE 1B (SOLUTION)

Which of the following interpretations about the class diagrams shown below is TRUE?



- a. Each borrower must have at least one book issued
- b. Each book may have less than one borrower**
- c. Each borrower may have 6 books issued to their name
- d. Each book must have at least one borrower

EXERCISE 2A

- Draw a class diagram showing the relationship between the two classes defined in the following code.

```
package library;

public class Book {
    private Borrower borrowedBy;

    public Book() {
        // Null indicates not on loan.
        borrowedBy = null;
    }

    boolean checkout( Borrower borrower ) {
        boolean success = false;
        if( borrowedBy == null ) {
            // Book isn't already on loan.
            borrowedBy = borrower;
            success = true;
        }
        return success;
    }
    ...
}
```

```
package library;

public class Borrower {
    private static final int BORROWING_LIMIT = 10;
    private List< Book > booksOnLoan;

    public Borrower() {
        booksOnLoan = new ArrayList< Book >( );
    }

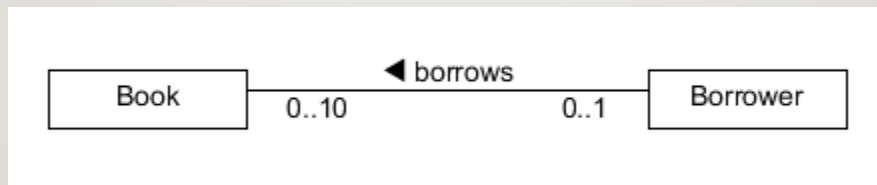
    public boolean borrow( Book book ) {
        if( book.isOnLoan( ) )
            return false; // Book is out to someone else.

        if( booksOnLoan.size( ) == BORROWING_LIMIT )
            return false; // Borrowing limit reached.

        booksOnLoan.add( book );
        book.checkout( this );
        return true;
    }
}
```

EXERCISE 2A (SOLUTION)

- Draw a class diagram showing the relationship between the two classes defined in the following code.



EXERCISE 2B

Which of the following object-oriented concepts are demonstrated in the following code?

- Polymorphism
- Inheritance
- Information hiding
- Encapsulation

```
package library;

public class Book {
    private Borrower borrowedBy;

    public Book() {
        // Null indicates not on loan.
        borrowedBy = null;
    }

    boolean checkout( Borrower borrower ) {
        boolean success = false;
        if( borrowedBy == null ) {
            // Book isn't already on loan.
            borrowedBy = borrower;
            success = true;
        }
        return success;
    }
    ...
}
```

```
package library;

public class Borrower {
    private static final int BORROWING_LIMIT = 10;
    private List< Book > booksOnLoan;

    public Borrower() {
        booksOnLoan = new ArrayList< Book >( );
    }

    public boolean borrow( Book book ) {
        if( book.isOnLoan() )
            return false; // Book is out to someone else.

        if( booksOnLoan.size() == BORROWING_LIMIT )
            return false; // Borrowing limit reached.

        booksOnLoan.add( book );
        book.checkout( this );
        return true;
    }
}
```

EXERCISE 2B

Which of the following object-oriented concepts are demonstrated in the following code?

Explain **how**?

```
package library;

public class Book {
    private Borrower borrowedBy;

    public Book() {
        // Null indicates not on loan.
        borrowedBy = null;
    }

    boolean checkout( Borrower borrower ) {
        boolean success = false;
        if( borrowedBy == null ) {
            // Book isn't already on loan.
            borrowedBy = borrower;
            success = true;
        }
        return success;
    }
    ...
}
```

```
package library;

public class Borrower {
    private static final int BORROWING_LIMIT = 10;
    private List< Book > booksOnLoan;

    public Borrower() {
        booksOnLoan = new ArrayList< Book >( );
    }

    public boolean borrow( Book book ) {
        if( book.isOnLoan() )
            return false; // Book is out to someone else.

        if( booksOnLoan.size() == BORROWING_LIMIT )
            return false; // Borrowing limit reached.

        booksOnLoan.add( book );
        book.checkout( this );
        return true;
    }
}
```

EXERCISE 2B (SOLUTION)

Which of the following object-oriented concepts are demonstrated in the following code?

Information hiding

- what happens when connecting the objects by calling **Borrower**'s borrow method.
- what happens when we call **Book**'s checkout method directly.
- how code outside of package library that makes the direct call **checkout()** call – won't compile.
- code outside of the library package cannot establish connectivity (incorrectly) by calling **Book's checkout()** method.
- This is a form of information hiding and is implemented by visibility qualifiers.

Encapsulation

- **Book** and **Borrower** both classes are encapsulations (describes the idea of bundling data and methods that work on that data within one unit) of their relevant data (fields) and procedural (methods) abstractions

SUGGESTED TASKS

- Develop the **UML model for Shapes application** to additionally include an object diagram and a sequence diagram.
 - The object diagram should include an instance of AnimationViewer, Timer, plus a couple of Shapes
 - The sequence diagram should show how objects interact in response to the timer ticking
 - The model as a whole should be well-formed
- For more practice:
 - Develop the UML model for A2 code too.
 - Solve Practice and Lab exercises

REVIEW

- **Learnt something about:**
 - The what and why of modelling
 - Models are abstractions of complex systems; they expose significant information and suppress irrelevant detail
 - Models help us to capture design decisions, to understand existing systems and to communicate design knowledge to others
 - The UML – the industry standard notation for modelling software-intensive systems
 - The UML includes diagram notations for modelling different aspects of software systems including static and dynamic structures (using class and object interaction diagrams respectively)
 - Models should be well-formed – individual views should be consistent with one another, resulting in a coherent model
 - UML: https://en.wikipedia.org/wiki/Unified_Modeling_Language

REFERENCES

➤ [Books]

- Chapter-5 Modelling with Classes, Timothy Lethbridge and Robert Leganaire, Object-Oriented Software Engineering: Practical Software Development using UML and Java, McGraw-Hill publications
- *Java How to Program*, Dietel and Dietel Publications

➤ [Online]

- Canvas
 - A2 code
- Unified Modelling Language
 - https://en.wikipedia.org/wiki/Unified_Modeling_Language

UP NEXT...

- **Object-oriented design objectives and principles**